

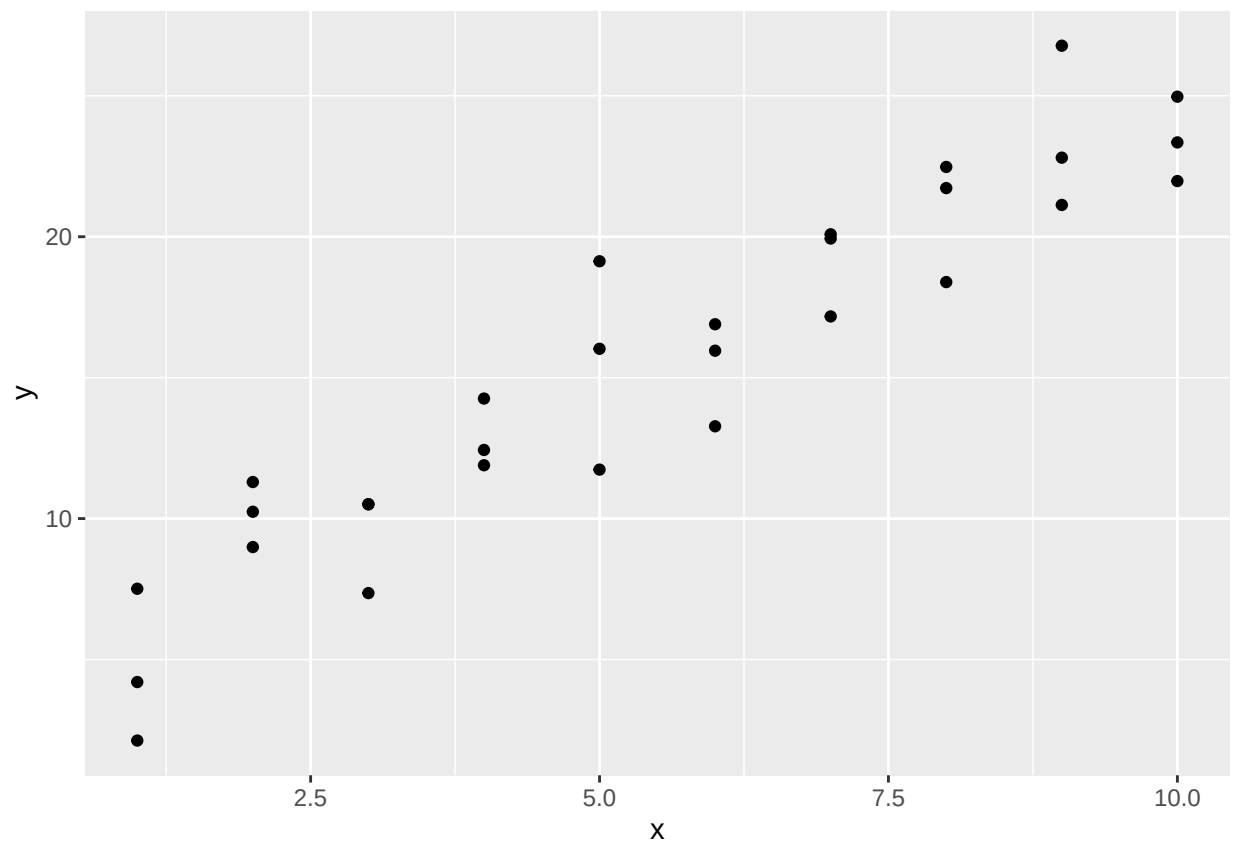
# Stat Models

2025-12-15

```
suppressMessages(library(tidyverse))  
library(modelr)  
library(gapminder)
```

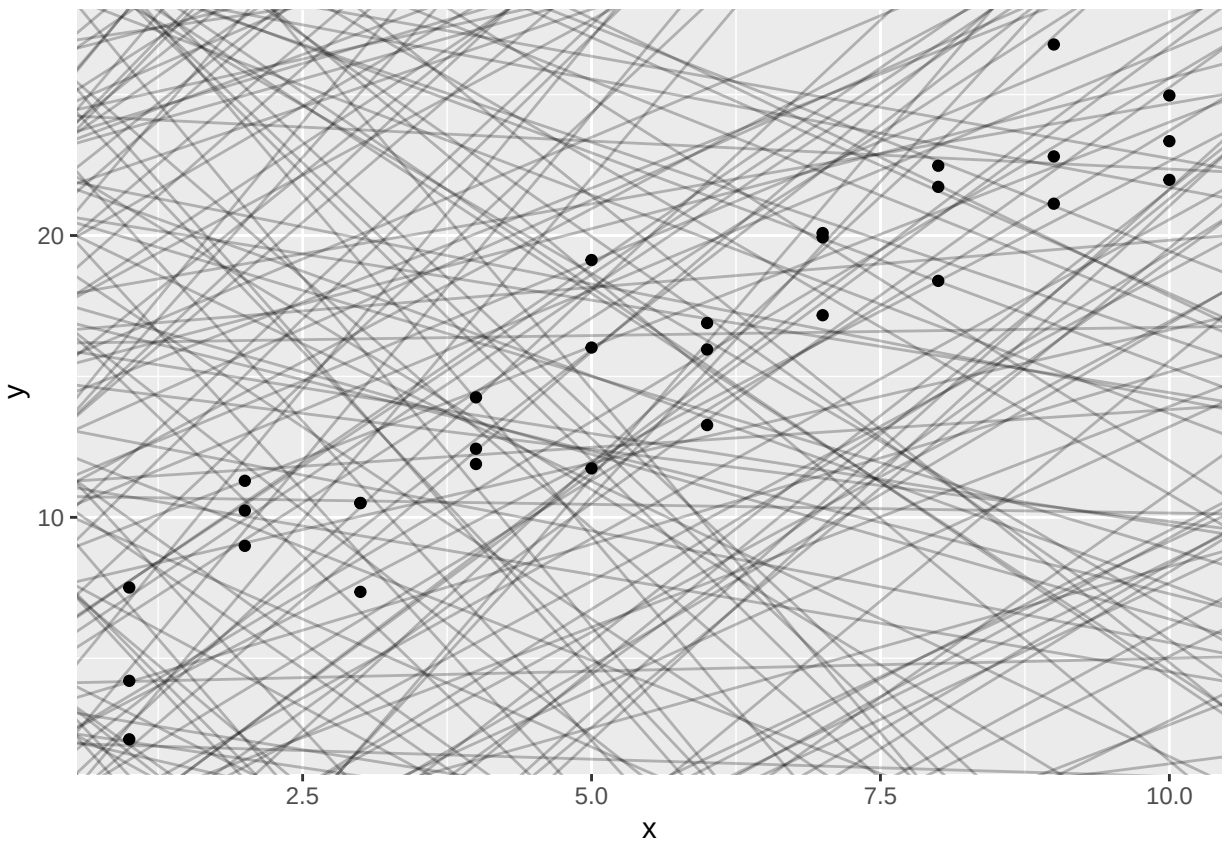
## A simple model (linear)

```
ggplot(sim1, aes(x, y)) +  
  geom_point()
```



```
## The Model  
models <- tibble(  
  a1 = runif(250, -20, 40), ## Run 250 models  
  a2 = runif(250, -5, 5)
```

```
)
ggplot(sim1, aes(x, y)) +
  geom_abline(aes(intercept = a1, slope = a2), data = models, alpha = 1/4) +
  geom_point()
```



Finding Residuals (This distance is the prediction and the actual y value in the data)

```
modell1 <- function(a, data) {
  a[1] + data$x * a[2]
}
modell1(c(7, 1.5), sim1)
```

```
## [1] 8.5 8.5 8.5 10.0 10.0 10.0 11.5 11.5 11.5 13.0 13.0 13.0 14.5 14.5 14.5
## [16] 16.0 16.0 16.0 17.5 17.5 17.5 19.0 19.0 19.0 20.5 20.5 20.5 22.0 22.0 22.0
```

Root square mean for ANOVA

```
measure_distance <- function(mod, data) {
  diff <- data$y - model1(mod, data)
  sqrt(mean(diff ^ 2))
}
measure_distance(c(7, 1.5), sim1)
```

```
## [1] 2.665212
```

Use purrr to compute the distance for all the models defined above.

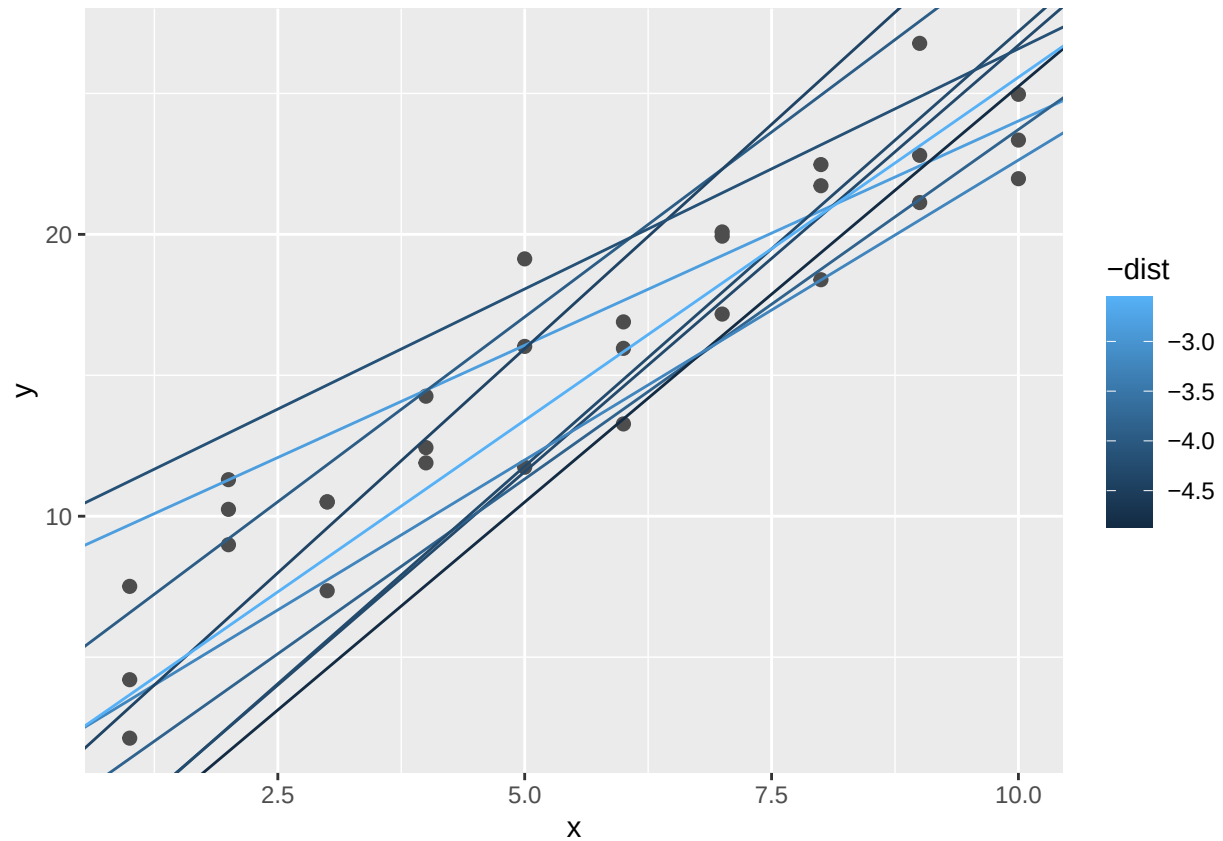
```
sim1_dist <- function(a1, a2) {
  measure_distance(c(a1, a2), sim1)
}

models <- models %>%
  mutate(dist = purrr::map2_dbl(a1, a2, sim1_dist))
models
```

```
## # A tibble: 250 x 3
##       a1    a2  dist
##   <dbl> <dbl> <dbl>
## 1  33.6  -4.86  21.7
## 2  -2.07  2.26   5.58
## 3  30.9  -4.96  23.5
## 4 -12.5   4.79   8.33
## 5  11.6   3.84  18.0
## 6  13.0  -1.61  15.6
## 7 -16.9   3.79  12.7
## 8  11.5   5.00  25.1
## 9 -18.4  -1.10  41.0
## 10 35.8   1.26  27.4
## # i 240 more rows
```

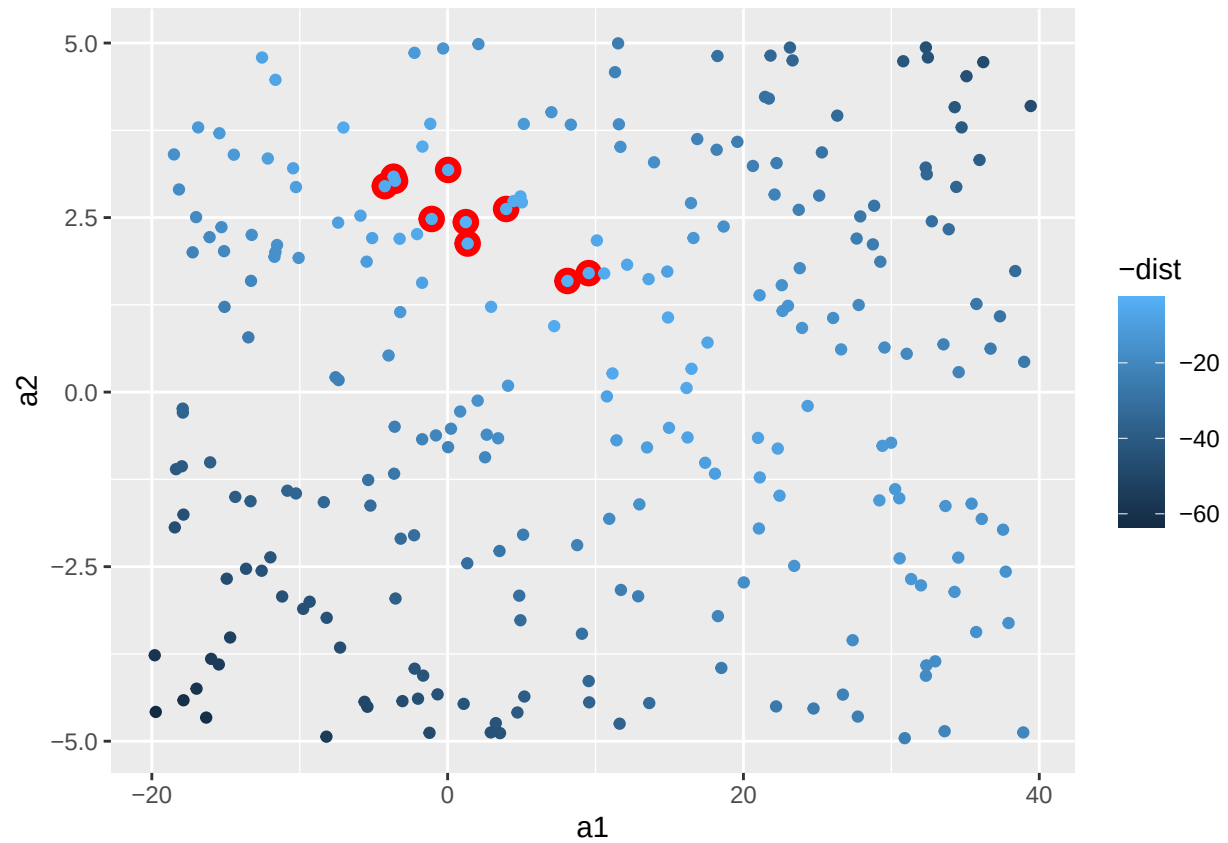
Best Model means smallest distance between predicted and observed

```
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, colour = "grey30") +
  geom_abline(
    aes(intercept = a1, slope = a2, colour = -dist),
    data = filter(models, rank(dist) <= 10)
  )
```



We could represent as a scatterplot (with loss of 0 being the best)

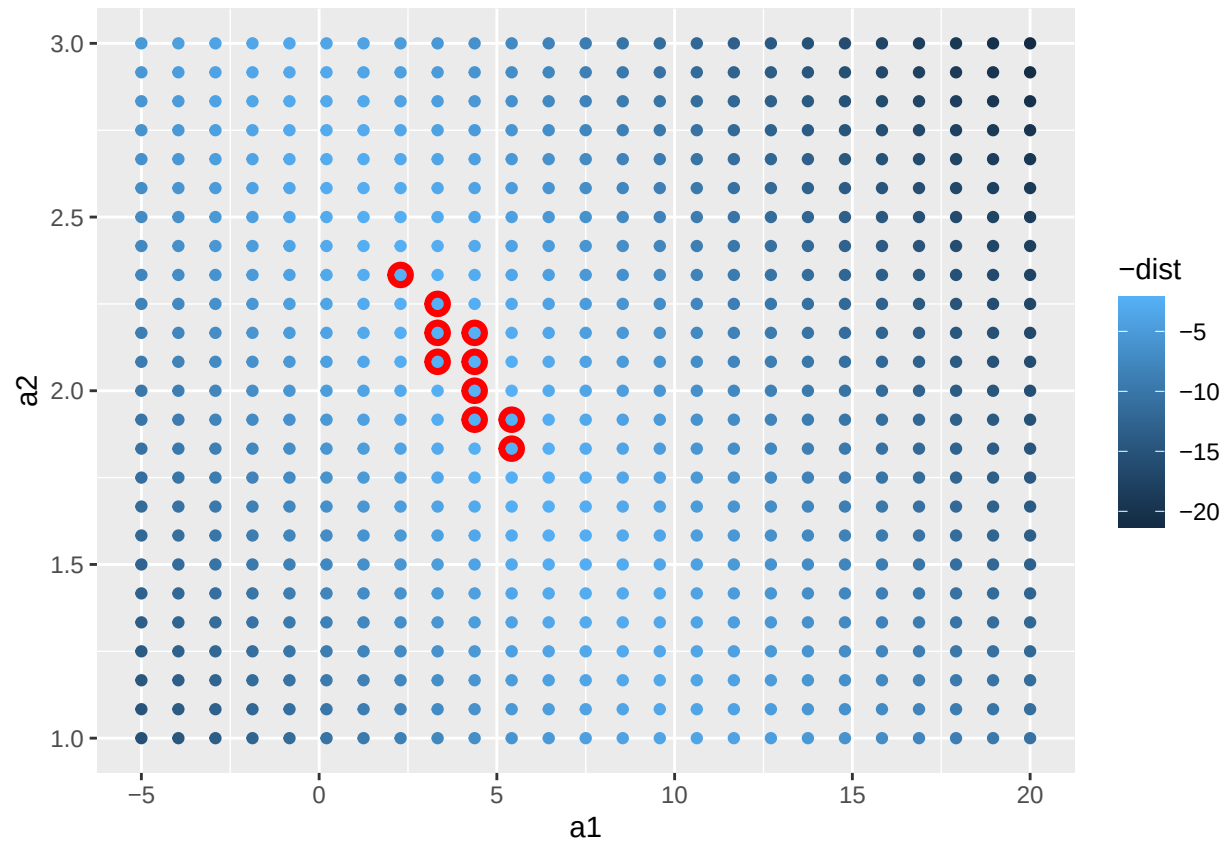
```
ggplot(models, aes(a1, a2)) +  
  geom_point(data = filter(models, rank(dist) <= 10), size = 4, colour = "red") +  
  geom_point(aes(colour = -dist))
```



## Conducting a grid search

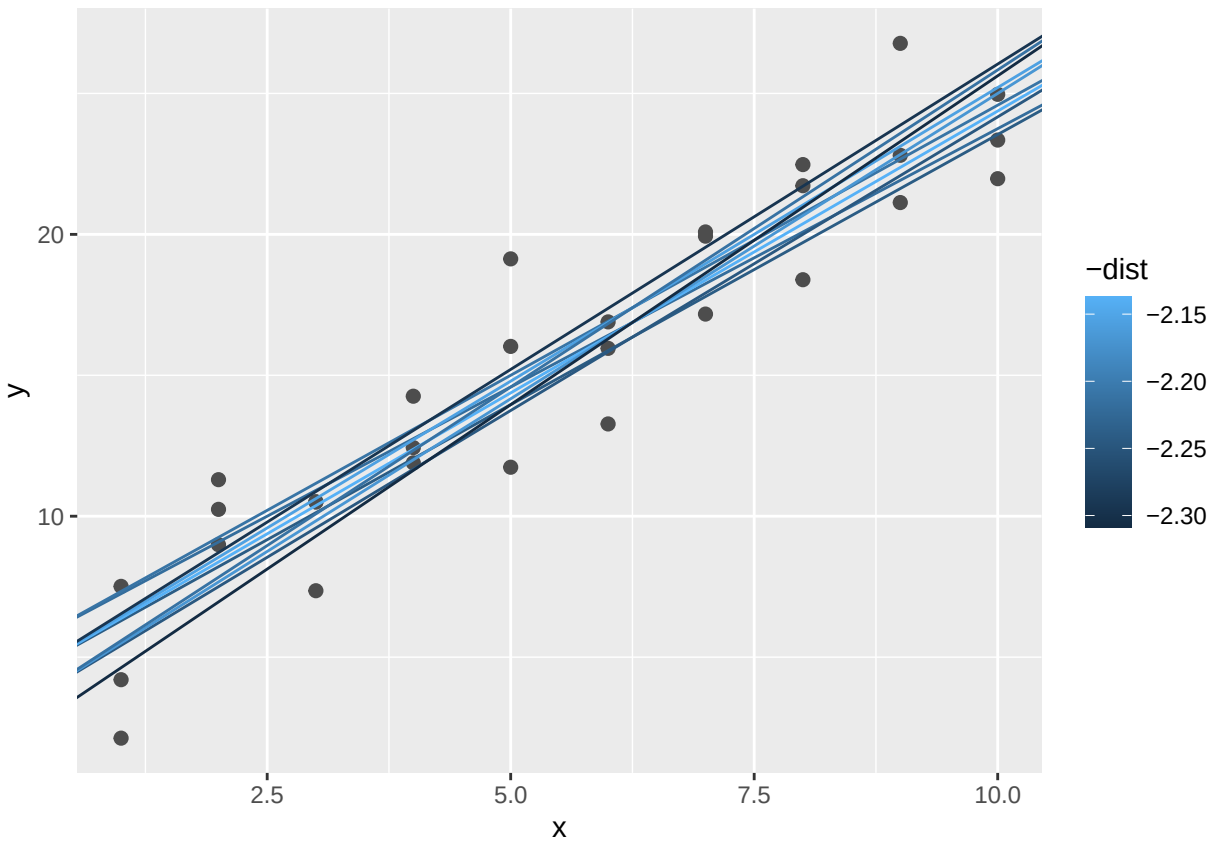
```
grid <- expand.grid(
  a1 = seq(-5, 20, length = 25),
  a2 = seq(1, 3, length = 25)
) %>%
  mutate(dist = purrr::map2_dbl(a1, a2, sim1_dist))

grid %>%
  ggplot(aes(a1, a2)) +
  geom_point(data = filter(grid, rank(dist) <= 10), size = 4, colour = "red") +
  geom_point(aes(colour = -dist))
```



## How to choose which model

```
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, colour = "grey30") +
  geom_abline(
    aes(intercept = a1, slope = a2, colour = -dist),
    data = filter(grid, rank(dist) <= 10)
  )
```

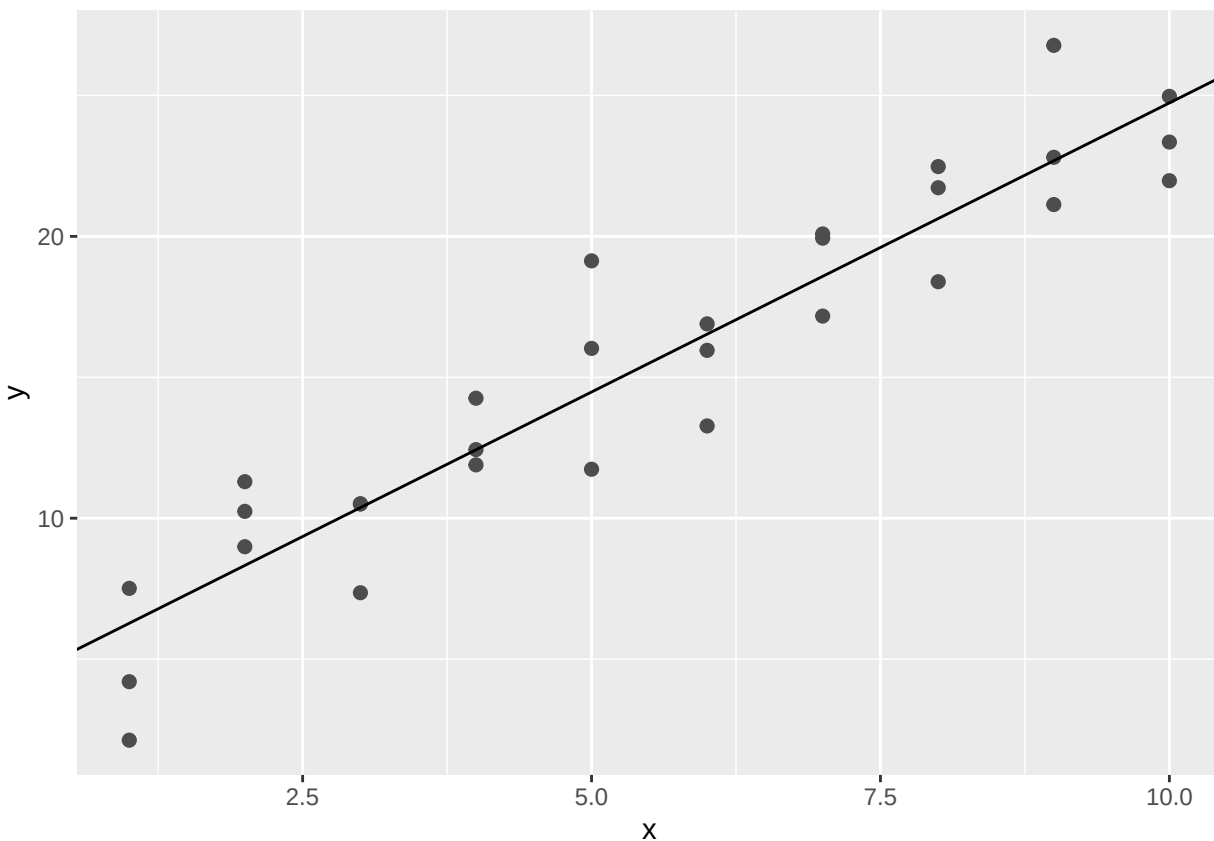


Iteratively making the grid finer and finer until you narrowed in on the best model.

```
best <- optim(c(0, 0), measure_distance, data = sim1)
best$par ## you pick a starting point and look around for the steepest slope.
```

```
## [1] 4.222248 2.051204
```

```
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, colour = "grey30") +
  geom_abline(intercept = best$par[1], slope = best$par[2])
```



## Fitting a linear model

```
sim1_mod <- lm(y ~ x, data = sim1)
coef(sim1_mod)
```

```
## (Intercept)          x
##    4.220822    2.051533
```

First argument is a data frame, and for each subsequent argument it finds the unique variables and then generates all combinations

```
grid <- sim1 %>%
  data_grid(x)
grid
```

```
## # A tibble: 10 x 1
##       x
##   <int>
## 1     1
## 2     2
## 3     3
```

```
## 4      4
## 5      5
## 6      6
## 7      7
## 8      8
## 9      9
## 10     10
```

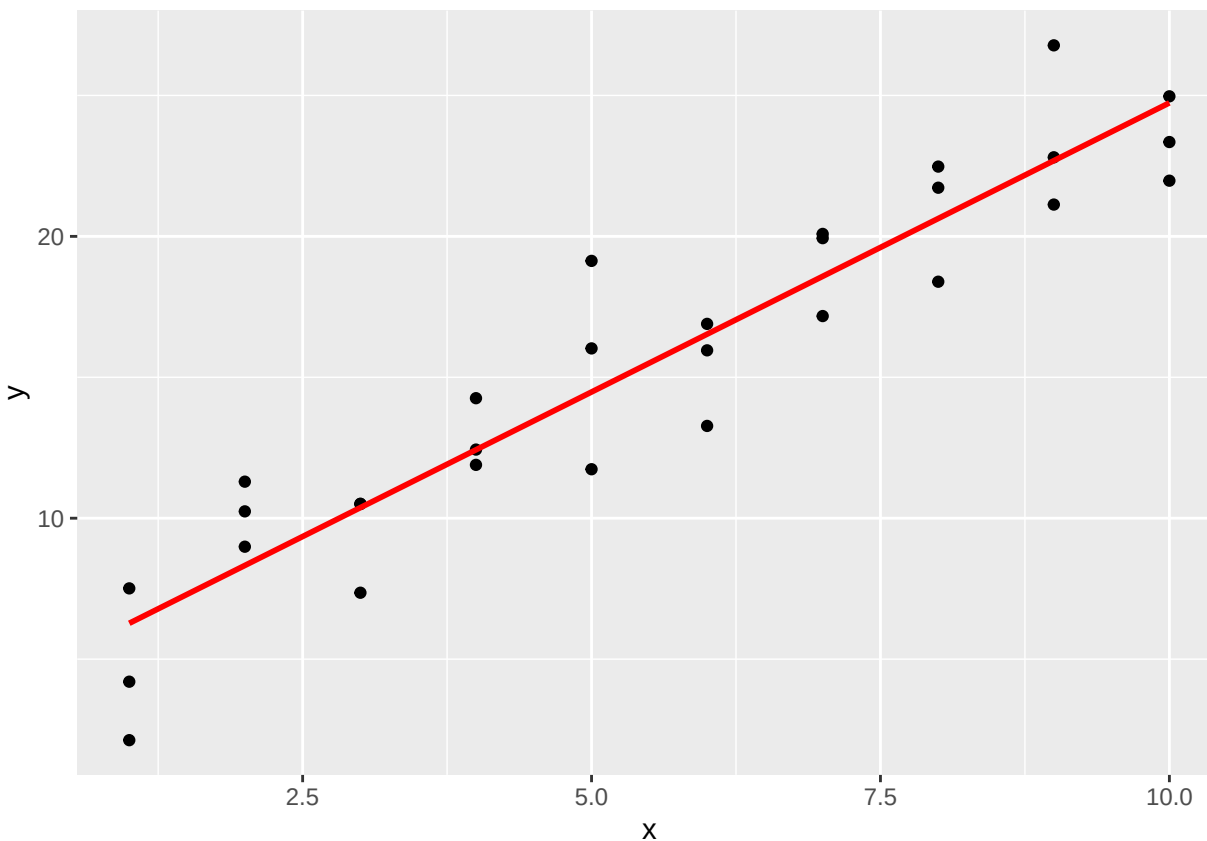
```
grid <- grid %>%
  add_predictions(sim1_mod) ## add the predictions
grid
```

```
## # A tibble: 10 x 2
##       x  pred
##   <int> <dbl>
## 1     1  6.27
## 2     2  8.32
## 3     3 10.4
## 4     4 12.4
## 5     5 14.5
## 6     6 16.5
## 7     7 18.6
## 8     8 20.6
## 9     9 22.7
## 10    10 24.7
```

Add the plot line (predictions)

```
ggplot(sim1, aes(x)) +
  geom_point(aes(y = y)) +
  geom_line(aes(y = pred), data = grid, colour = "red", size = 1)
```

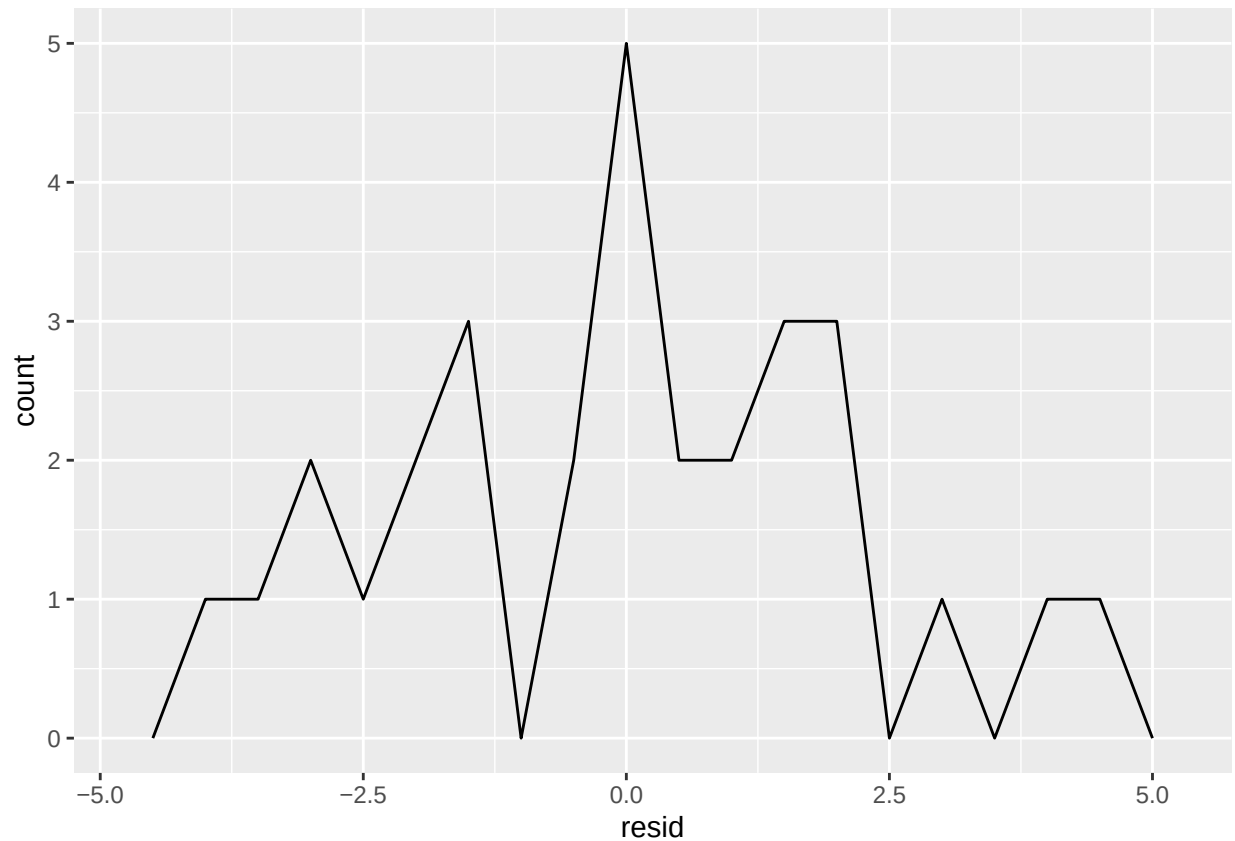
```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



```
sim1 <- sim1 %>%
  add_residuals(sim1_mod)
sim1
```

```
## # A tibble: 30 x 3
##       x     y   resid
##   <int> <dbl> <dbl>
## 1     1  4.20 -2.07
## 2     1  7.51  1.24
## 3     1  2.13 -4.15
## 4     2  8.99  0.665
## 5     2 10.2   1.92
## 6     2 11.3   2.97
## 7     3  7.36 -3.02
## 8     3 10.5   0.130
## 9     3 10.5   0.136
## 10    4 12.4   0.00763
## # i 20 more rows
```

```
ggplot(sim1, aes(resid)) +
  geom_freqpoly(binwidth = 0.5)
```



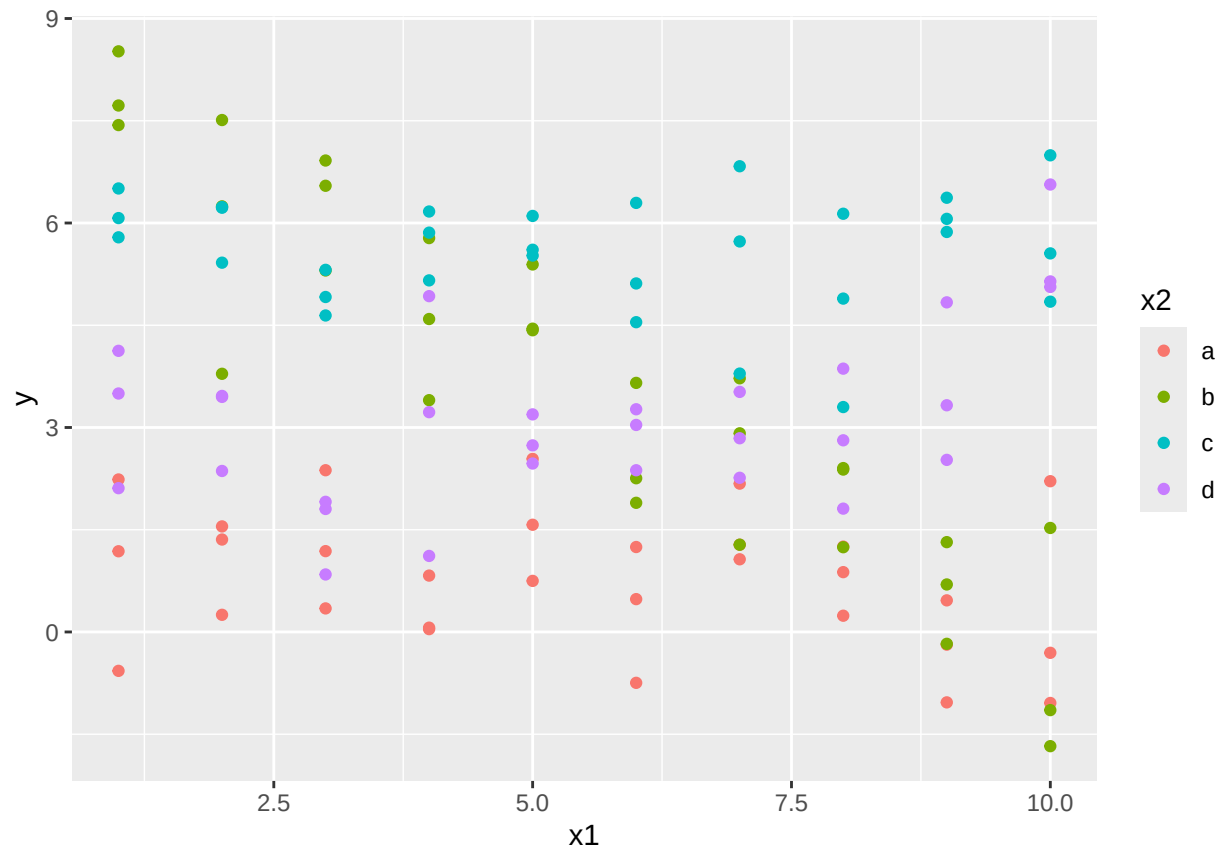
One-hot encoding (converting categorical to continuous using binary)

```
df <- tribble(
  ~ sex, ~ response,
  "male", 1,
  "female", 2,
  "male", 1
)
model_matrix(df, response ~ sex)
```

```
## # A tibble: 3 x 2
##   '(Intercept)' sexmale
##           <dbl>   <dbl>
## 1             1       1
## 2             1       0
## 3             1       1
```

Categorical predictor and continuous predictor

```
ggplot(sim3, aes(x1, y)) +
  geom_point(aes(colour = x2))
```



```
mod1 <- lm(y ~ x1 + x2, data = sim3)
mod2 <- lm(y ~ x1 * x2, data = sim3) ## interaction
```

## Interactions (two continuous)

```
mod1 <- lm(y ~ x1 + x2, data = sim4)
mod2 <- lm(y ~ x1 * x2, data = sim4)
```

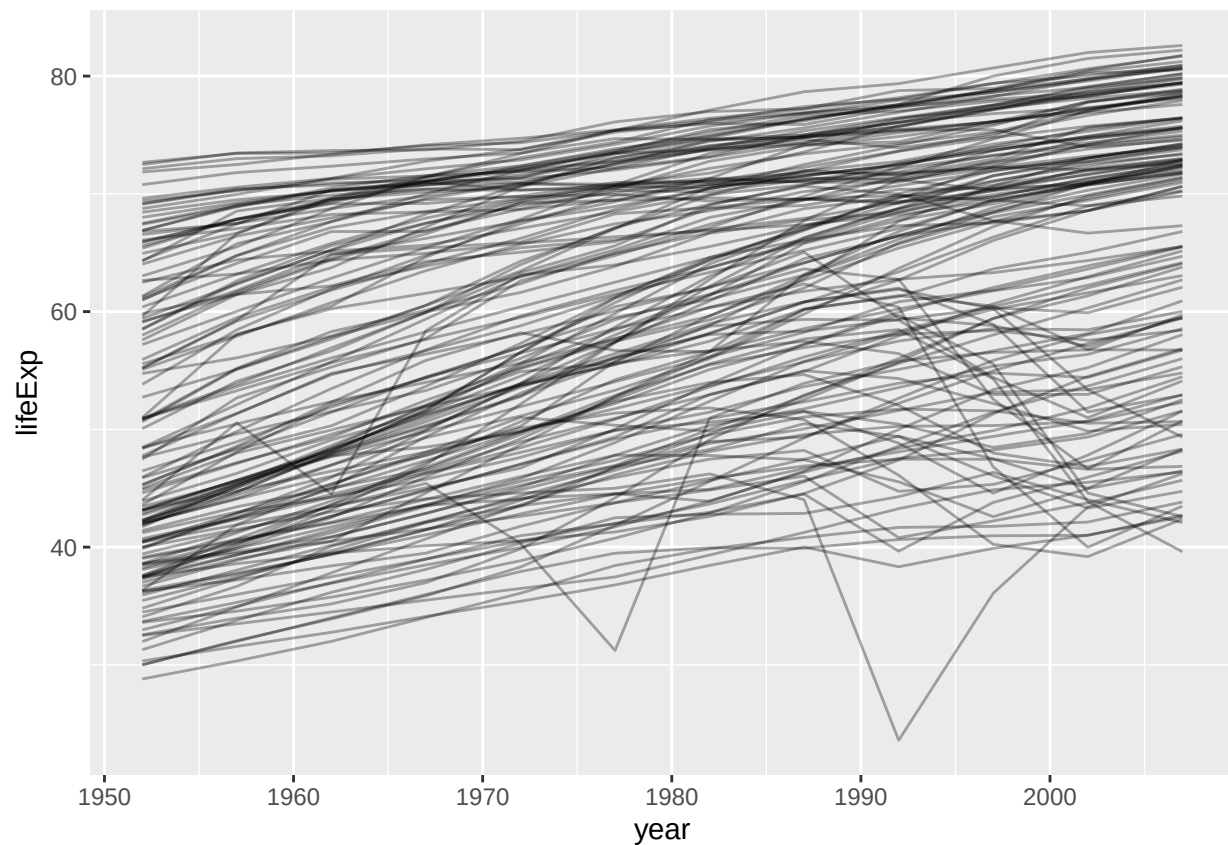
```
grid <- sim4 %>%
  data_grid(
    x1 = seq_range(x1, 5),
    x2 = seq_range(x2, 5)
  ) %>%
  gather_predictions(mod1, mod2)
grid
```

```
## # A tibble: 50 x 4
##   model  x1  x2  pred
##   <chr> <dbl> <dbl> <dbl>
## 1 mod1  -1   -1  0.996
## 2 mod1  -1  -0.5 -0.395
## 3 mod1  -1    0 -1.79
## 4 mod1  -1    0.5 -3.18
```

```
## 5 mod1 -1 1 -4.57
## 6 mod1 -0.5 -1 1.91
## 7 mod1 -0.5 -0.5 0.516
## 8 mod1 -0.5 0 -0.875
## 9 mod1 -0.5 0.5 -2.27
## 10 mod1 -0.5 1 -3.66
## # i 40 more rows
```

## Graphing

```
gapminder %>%
  ggplot(aes(year, lifeExp, group = country)) +
  geom_line(alpha = 1/3)
```



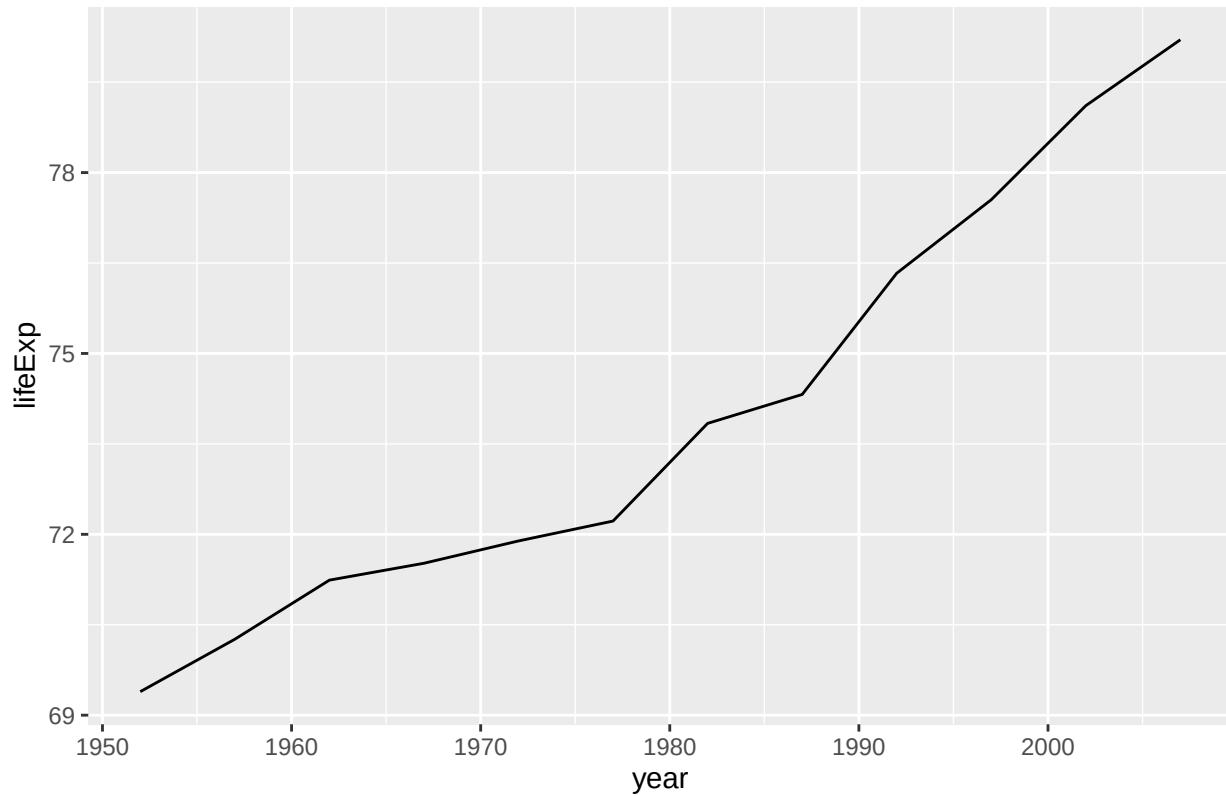
## Fit the model

```
country_model <- function(df) {
  lm(lifeExp ~ year, data = df)
}
```

## Model Quality

```
nz <- filter(gapminder, country == "New Zealand")
nz %>%
  ggplot(aes(year, lifeExp)) +
  geom_line() +
  ggtitle("Full data = ")
```

Full data =

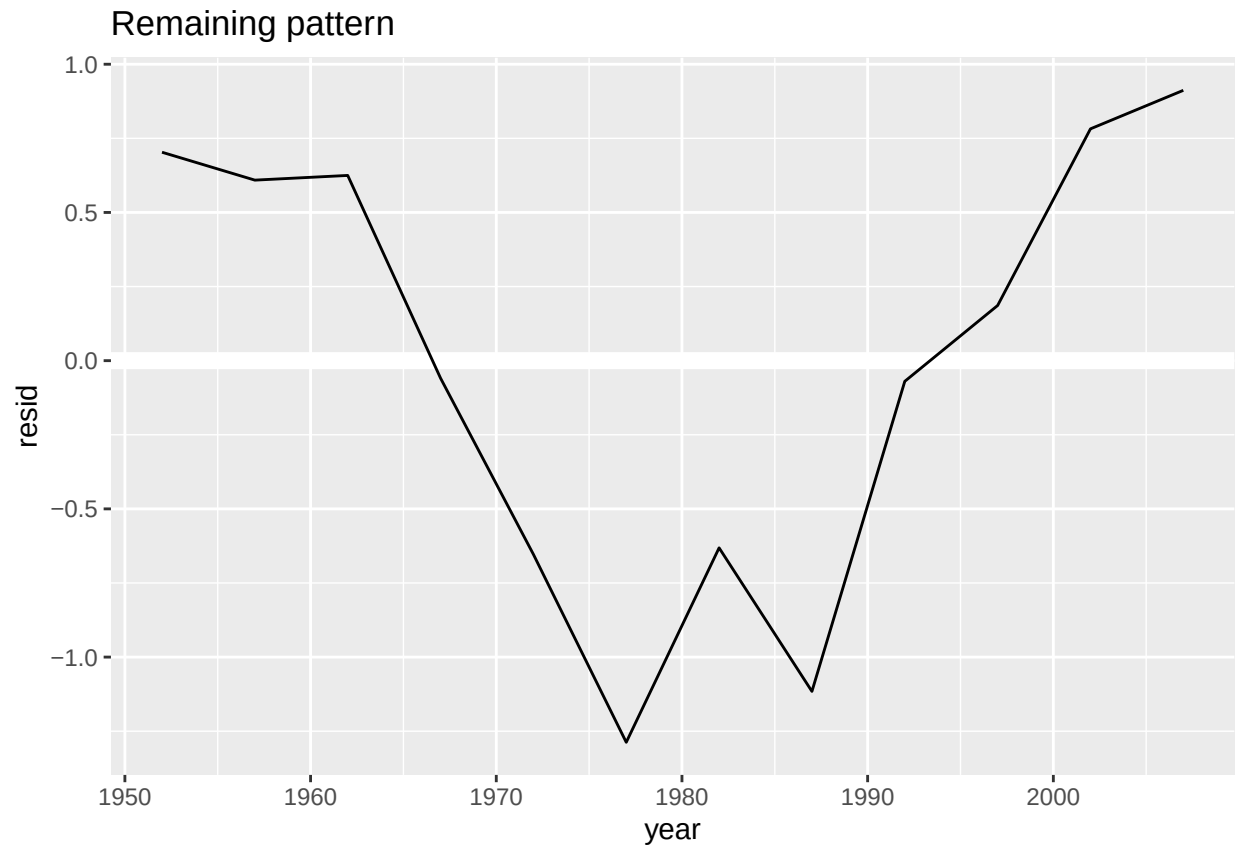


```
nz_mod <- lm(lifeExp ~ year, data = nz)
nz %>%
  add_predictions(nz_mod) %>%
  ggplot(aes(year, pred)) +
  geom_line() +
  ggtitle("Linear trend + ")
```

### Linear trend +



```
nz %>%  
  add_residuals(nz_mod) %>%  
  ggplot(aes(year, resid)) +  
  geom_hline(yintercept = 0, colour = "white", size = 3) +  
  geom_line() +  
  ggtitle("Remaining pattern")
```



## Nested data

```
by_country <- gapminder %>%
  group_by(country, continent) %>%
  nest()
```

```
by_country
```

```
## # A tibble: 142 x 3
## # Groups:   country, continent [142]
##   country    continent data
##   <fct>      <fct>    <list>
## 1 Afghanistan Asia      <tibble [12 x 4]>
## 2 Albania     Europe    <tibble [12 x 4]>
## 3 Algeria     Africa    <tibble [12 x 4]>
## 4 Angola      Africa    <tibble [12 x 4]>
## 5 Argentina   Americas  <tibble [12 x 4]>
## 6 Australia   Oceania   <tibble [12 x 4]>
## 7 Austria     Europe    <tibble [12 x 4]>
## 8 Bahrain     Asia      <tibble [12 x 4]>
## 9 Bangladesh  Asia      <tibble [12 x 4]>
## 10 Belgium    Europe    <tibble [12 x 4]>
## # i 132 more rows
```

```
## Load the data
gapminder
```

```
## # A tibble: 1,704 x 6
##   country    continent year lifeExp      pop gdpPercap
##   <fct>      <fct>    <int>  <dbl>    <int>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Afghanistan Asia      1957   30.3  9240934    821.
## 3 Afghanistan Asia      1962   32.0 10267083    853.
## 4 Afghanistan Asia      1967   34.0 11537966    836.
## 5 Afghanistan Asia      1972   36.1 13079460    740.
## 6 Afghanistan Asia      1977   38.4 14880372    786.
## 7 Afghanistan Asia      1982   39.9 12881816    978.
## 8 Afghanistan Asia      1987   40.8 13867957    852.
## 9 Afghanistan Asia      1992   41.7 16317921    649.
## 10 Afghanistan Asia      1997   41.8 22227415    635.
## # i 1,694 more rows
```

```
broom::glance(nz_mod)
```

```
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic      p.value    df logLik   AIC   BIC
##   <dbl>      <dbl> <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl>
## 1    0.954      0.949 0.804    205. 0.0000000541     1  -13.3  32.6  34.1
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

```
models <- map(by_country$data, country_model)
```

```
by_country <- by_country %>%
  mutate(model = map(data, country_model))
by_country
```

```
## # A tibble: 142 x 4
## # Groups:   country, continent [142]
##   country    continent data          model
##   <fct>      <fct>    <list>      <list>
## 1 Afghanistan Asia      <tibble [12 x 4]> <lm>
## 2 Albania     Europe    <tibble [12 x 4]> <lm>
## 3 Algeria      Africa    <tibble [12 x 4]> <lm>
## 4 Angola        Africa    <tibble [12 x 4]> <lm>
## 5 Argentina    Americas <tibble [12 x 4]> <lm>
## 6 Australia    Oceania   <tibble [12 x 4]> <lm>
## 7 Austria       Europe    <tibble [12 x 4]> <lm>
## 8 Bahrain       Asia      <tibble [12 x 4]> <lm>
## 9 Bangladesh   Asia      <tibble [12 x 4]> <lm>
## 10 Belgium      Europe    <tibble [12 x 4]> <lm>
## # i 132 more rows
```

```
by_country %>%
  mutate(glance = map(model, broom::glance)) %>%
  unnest(glance)
```

```
## # A tibble: 142 x 16
## # Groups:   country, continent [142]
##   country    continent data      model  r.squared adj.r.squared sigma statistic
##   <fct>      <fct>    <list>   <list>    <dbl>      <dbl> <dbl>      <dbl>
## 1 Afghanistan Asia      <tibble> <lm>      0.948      0.942 1.22      181.
## 2 Albania     Europe    <tibble> <lm>      0.911      0.902 1.98      102.
## 3 Algeria     Africa    <tibble> <lm>      0.985      0.984 1.32      662.
## 4 Angola      Africa    <tibble> <lm>      0.888      0.877 1.41       79.1
## 5 Argentina   Americas <tibble> <lm>      0.996      0.995 0.292     2246.
## 6 Australia   Oceania   <tibble> <lm>      0.980      0.978 0.621      481.
## 7 Austria     Europe    <tibble> <lm>      0.992      0.991 0.407     1261.
## 8 Bahrain     Asia      <tibble> <lm>      0.967      0.963 1.64       291.
## 9 Bangladesh  Asia      <tibble> <lm>      0.989      0.988 0.977      930.
## 10 Belgium    Europe    <tibble> <lm>      0.995      0.994 0.293     1822.
## # i 132 more rows
## # i 8 more variables: p.value <dbl>, df <dbl>, logLik <dbl>, AIC <dbl>,
## #   BIC <dbl>, deviance <dbl>, df.residual <int>, nobs <int>
```

## Evaluating Model Fit

```
glance <- by_country %>%
  mutate(glance = map(model, broom::glance)) %>%
  unnest(glance, .drop = TRUE)
```

```
## Warning: The 'drop' argument of 'unnest()' is deprecated as of tidyr 1.0.0.
## i All list-columns are now preserved.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
glance %>%
  arrange(r.squared)
```

```
## # A tibble: 142 x 16
## # Groups:   country, continent [142]
##   country    continent data      model  r.squared adj.r.squared sigma statistic
##   <fct>      <fct>    <list>   <lis>    <dbl>      <dbl> <dbl>      <dbl>
## 1 Rwanda     Africa    <tibble> <lm>      0.0172    -0.0811 6.56      0.175
## 2 Botswana   Africa    <tibble> <lm>      0.0340    -0.0626 6.11      0.352
## 3 Zimbabwe   Africa    <tibble> <lm>      0.0562    -0.0381 7.21      0.596
## 4 Zambia     Africa    <tibble> <lm>      0.0598    -0.0342 4.53      0.636
## 5 Swaziland  Africa    <tibble> <lm>      0.0682    -0.0250 6.64      0.732
## 6 Lesotho    Africa    <tibble> <lm>      0.0849    -0.00666 5.93      0.927
## 7 Cote d'Ivoi~ Africa    <tibble> <lm>      0.283     0.212 3.93      3.95
## 8 South Africa Africa    <tibble> <lm>      0.312     0.244 4.74      4.54
## 9 Uganda     Africa    <tibble> <lm>      0.342     0.276 3.19      5.20
## 10 Congo, Dem.~ Africa    <tibble> <lm>      0.348     0.283 2.43      5.34
## # i 132 more rows
## # i 8 more variables: p.value <dbl>, df <dbl>, logLik <dbl>, AIC <dbl>,
## #   BIC <dbl>, deviance <dbl>, df.residual <int>, nobs <int>
```

## Keeps columns together

```
gapminder %>%  
  group_by(country, continent) %>%  
  nest()
```

```
## # A tibble: 142 x 3  
## # Groups:   country, continent [142]  
##   country      continent data  
##   <fct>        <fct>    <list>  
## 1 Afghanistan Asia      <tibble [12 x 4]>  
## 2 Albania      Europe   <tibble [12 x 4]>  
## 3 Algeria      Africa   <tibble [12 x 4]>  
## 4 Angola       Africa   <tibble [12 x 4]>  
## 5 Argentina    Americas <tibble [12 x 4]>  
## 6 Australia    Oceania  <tibble [12 x 4]>  
## 7 Austria      Europe   <tibble [12 x 4]>  
## 8 Bahrain      Asia     <tibble [12 x 4]>  
## 9 Bangladesh   Asia     <tibble [12 x 4]>  
## 10 Belgium     Europe   <tibble [12 x 4]>  
## # i 132 more rows
```